SPY Multi Agent Trading System

Midterm AI Multi-Agent Trading Challenge

Author(s):

Mohammad Haj (mhaj2@illinois.edu)
Varad Gandhi (vgandhi3@illinois.edu)
Harsh Hari (harsh6@illinois.edu)
Pavithra Vaitheeswaran (pvaith2@illinois.edu)
Mike Feistel (feistel2@illinois.edu)
Ming Chia Tsai (mt74@illinois.edu)

Date: March 2025

Executive Summary

This report documents the development of an AI-driven **multi-agent trading system for the SPDR S&P 500 ETF (SPY)**. The system integrates multiple analytical "agents" – including technical indicators, news sentiment, and economic indicators – to generate trading signals. The trading strategy operates on a weekly cycle, entering either long or short positions in SPY at the start of each week based on a consensus of these agents' signals, with **strict risk management** (fixed position sizing and stop-loss rules). Backtesting over the year-long period (2024 into early 2025) demonstrates that the strategy achieved **steady, low-volatility growth** of the portfolio. Key performance metrics include a **Sharpe Ratio ~1.19**, **Total Return ~+6.75%**, **Max Drawdown ~1.9%**, and a **Win Rate ~53.6%**. Notably, while the strategy's absolute return trailed the strong buy-and-hold performance of SPY (which returned nearly 30% in that period), it maintained far lower drawdowns and volatility, highlighting its focus on **capital preservation and risk-adjusted returns**. The multi-agent approach provided balanced signals that helped avoid major losses (no single trade risked more than ~1.5% of capital), though it sometimes caused the system to miss out on large trend gains. Overall, the project demonstrates how combining diverse indicators can create a robust trading strategy, and it offers insights into improving such a system to better capture upside while still limiting risk.

Introduction

The purpose of this project is to design and evaluate an AI-powered **multi-agent trading strategy for the SPY ETF**. SPY is an exchange-traded fund tracking the S&P 500 index, widely traded for its liquidity and representation of the broad U.S. equity market. Given SPY's significance, the trading goal is to create a system that can **consistently profit from market movements** while **managing risk**, using an ensemble of signals from different analytical domains (technical analysis, news sentiment, macroeconomic trends). This

midterm challenge project aims to simulate a real-world quantitative trading approach: data from various sources are collected and processed by dedicated "agent" modules, and their outputs collectively inform trading decisions. We seek to leverage these multiple inputs to navigate market conditions more intelligently than any single indicator alone, ideally achieving a high risk-adjusted return (e.g. Sharpe Ratio) and minimal drawdowns. The following sections detail the system's design, data collection, strategy logic, backtesting results, and findings.

System Design Overview

The trading system is implemented in a **modular architecture**, separating responsibilities into distinct components for clarity and maintainability. The design is based on two primary Jupyter notebooks (agents) and a dashboard script:

- **Data Collection Agent**: Responsible for gathering and preprocessing all necessary data (price history, technical indicators, news, economic data). This module operates independently to fetch data from external sources (such as Yahoo Finance and Federal Reserve Economic Data) and compute derivative features. By isolating data handling, we ensure the strategy logic can be developed and tested with consistent, clean input data.

- **Strategy/Analysis Agent**: Responsible for generating trading signals using the processed data and executing a backtesting simulation. This notebook loads the cleaned datasets produced by the data agent, computes composite signals (the "multi-agent" decision), applies risk management rules, and simulates trades over time. The strategy agent is modular in itself – it includes sub-components or functions for each type of signal (technical, sentiment, economic) and a mechanism to combine them into final trade signals.

- **Dashboard (Streamlit) Module**: A separate script (dashboard.py) creates an interactive dashboard to visualize the performance of the strategy versus a benchmark. This dashboard reads the results of the backtest (e.g. daily portfolio values and summary metrics) and presents key performance indicators and charts for end-user review.

This modular approach offers several benefits. Each agent can be developed and tested in isolation – for example, data gathering can be updated or expanded without altering the strategy logic. It also mirrors a real multi-agent system where different agents (technical, news, econ) operate somewhat independently but feed into a central decision process. In terms of maintainability, having clearly defined separation means future improvements to one aspect (say adding a new indicator or using a new data source) can be integrated with minimal impact on other parts of the system.

Data Collection Module

The data collection module assembles a comprehensive dataset for SPY trading, pulling from **multiple sources** and ensuring the data is cleaned and aligned by date. The key data sources and steps include:

- **Price History (SPY and Related Markets)**: We obtained **daily OHLCV price data for SPY** over the past year using the Yahoo Finance API. In addition to SPY's own prices, related market indicators were gathered to enrich our analysis:
    - The **CBOE Volatility Index (VIX)**, which often inversely correlates with equity market moves (a proxy for market fear/uncertainty).
    - Sector ETFs like **XLK (Technology)** and **XLF (Financials)**, used to gauge sector rotation or risk appetite (tech vs. financial relative performance).
    - The **10-Year Treasury Yield** (constant maturity rate) from the Federal Reserve Economic Data (FRED) service, representing interest rate trends.

These series were merged on the date index with SPY's prices. After merging, any days with missing data (for example, if one series did not have a value due to market holidays or delayed starts) were handled by dropping those dates to maintain consistency (ensuring all inputs align in time). By dropping or forward-filling minor gaps, we avoided misaligned signals. The final price dataset includes daily columns for SPY's **Open, High, Low, Close, Volume** and additional columns for **VIX, XLK, XLF, and 10Y Yield**, each indexed by date.

- **Technical Indicators**: Using the price data, we computed classic technical indicators that serve as inputs to our strategy:
    - **Relative Strength Index (RSI)** – a momentum oscillator measuring the magnitude of recent price gains vs losses over a 14-day window. RSI values range from 0 to 100; readings below 30 indicate **oversold** conditions (potentially bullish reversal signals) and above 70 indicate **overbought** conditions (potential bearish reversal).
    - **Simple Moving Average (SMA)** – we calculated a 14-day SMA of SPY's closing price. The SMA smooths short-term fluctuations and helps identify trend direction; prices above the SMA might indicate an uptrend, below it a downtrend.
    - **Moving Average Convergence Divergence (MACD)** – a trend-following momentum indicator computed as the difference between a fast 12-day exponential moving average and a slower 26-day EMA of SPY's price, with a 9-day EMA of this difference forming the signal line. Positive MACD values signal upward momentum and negative values signal downward momentum; crossovers of the MACD and its signal line are often used as trade triggers.

These indicators were computed using a Python technical analysis library (pandas_ta) and added as new columns to the dataset. For instance, we appended columns "RSI", "SMA", and "MACD" to each date's record. The first few weeks of indicator values are NaN due to lookback periods (e.g., we need at least 14 days of data to compute a 14-day RSI), but by mid-January 2024 all indicators had valid values. We ensured any resulting missing values were dropped or forward-filled as needed so that by the time the strategy uses them, each trading day has a complete set of indicator inputs.

- **News Sentiment Data**: We incorporated a **news analysis agent** that processes daily financial news relevant to the market. The provided newsdata.csv contained news headlines and descriptions along with dates. The data collection agent parsed this file to derive a **daily sentiment score**. Each article's text was analyzed (e.g., using a sentiment analysis model or heuristic word scoring) to produce an article sentiment measure (where positive values indicate optimistic/bullish tone and negative values indicate pessimistic/bearish tone). These were then averaged (or summed) for each day to get a **daily sentiment_score** for the market. For example, if most news on a given day had negative tone, that day's sentiment_score would be negative. After computing sentiment_score per day, we generated a corresponding **news Trade_Signal**: if the sentiment_score > 0 (net positive sentiment), label it as "Buy"; otherwise label "Sell". This yielded a series of daily signals purely based on news sentiment, which we will later combine with other signals.

- **Economic Indicators**: Macroeconomic conditions can greatly influence equity markets, so we included four key economic indicators:

    - **Unemployment Rate (%)**
    - **Federal Funds Rate (%)** – proxy for interest rate policy
    - **Consumer Price Index (CPI)** – inflation measure
    - **Gross Domestic Product (GDP)** – economic growth measure (in billions of USD)

These indicators are typically reported monthly or quarterly. For the purpose of daily alignment, we interpolated or forward-filled values so that each trading day in our dataset has a value for these metrics. The Economicindicators.csv file provided a daily time series for these fields from 2020 through 2025, which appears to be an interpolation of actual releases (for example, unemployment might gradually change day-to-day in the dataset even though the real unemployment rate is reported monthly). We merged the latest values of these indicators into our main data by date. The data collection agent also computed changes where relevant (e.g., daily percentage change in the 10-year yield, captured as Yield_Change, or changes in the Tech vs Financials ratio) to use in sentiment or trade logic.

- **Data Cleaning & Preparation**: Throughout the data merging process, we took care to handle **missing values and outliers**. Missing data after merges were dropped to

avoid carrying forward misaligned days. For instance, when merging FRED's 10Y yield (which might not have data on weekends or certain holidays) with SPY (no weekends data either), any resulting NaN on a trading day would indicate a missing econ data point – such days were removed or that field was filled with the last known value if appropriate. Outliers in indicators like VIX (which can spike dramatically) or yield changes were implicitly handled by our strategy logic (e.g., a sudden VIX jump will trigger a strong bearish signal, but our position sizing and stop-loss will contain the risk of that event). We also normalized certain inputs if needed (notably, using percent changes for VIX and yields rather than raw differences, to have comparable scale in our sentiment signal logic). By the end of the data collection stage, we had a single consolidated DataFrame of daily data, indexed by Date, containing all relevant features: SPY prices, technical indicators (RSI, SMA, MACD), sentiment signals (news sentiment score or preliminary signal), and economic metrics.

In summary, the data collection agent produced a **clean, feature-rich dataset** for the strategy to consume. It encapsulated technical market trends, investor sentiment from news, and macroeconomic context – providing the "multi-agent" inputs needed for informed trading decisions.

Strategy Agent Module

The strategy agent takes the processed data and formulates a trading strategy by combining signals from the different "agents" (technical, sentiment, economic). This module can be thought of as the decision-maker that listens to each agent and decides when to buy or sell SPY. Below we outline the components of the strategy agent and how they influence trade entries/exits:

Technical Indicators & Signals: We selected a few proven technical indicators for the strategy rules, as introduced earlier (RSI, MACD, SMA). Their roles in the strategy are as follows:

- **RSI (14-day)**: We interpret RSI to identify **momentum reversals**. When RSI is very low (<30), SPY is considered oversold – a condition we consider as part of a *bullish* signal (price may rebound upwards). When RSI is very high (>70), SPY is overbought – considered as part of a *bearish* signal (price may correct downwards). Instead of trading on RSI alone, our strategy uses RSI as a filter/confirming indicator for other signals. For example, if another model predicts a price rise, we gain more confidence to go long if RSI is oversold (since a rebound is more likely). Conversely, a prediction of decline combined with overbought RSI strengthens a short signal.

- **MACD (12-26-9)**: We use the MACD indicator to capture **trend momentum**. A positive MACD value (fast EMA above slow EMA) indicates an uptrend, and negative value indicates a downtrend. In our system, MACD is one of the features fed into an ML-based predictor (described below) rather than having an explicit rule like

"MACD crossover = buy". However, conceptually, if MACD was strongly positive and rising, the model would likely lean bullish, and vice versa for negative MACD. MACD helps the strategy agent gauge whether momentum is on the side of a potential trade.

- **Simple Moving Average (14-day SMA)**: The SMA provides a short-term trend baseline. Our strategy's ML component uses the SMA value (and implicitly the relationship of price to SMA) as an input. Generally, if SPY's price is above the SMA and trending upward, it's supportive of bullish trades; if below the SMA, it's a bearish environment. We did not use a direct crossover rule, but the SMA's direction and gap from price influence the model's recommendation.

- **Bollinger Bands (20-day SMA ± 2σ)**: *While not implemented in the final code, Bollinger Bands are a notable technical tool we considered.* They consist of a moving average (usually 20-day) and upper/lower bands set at two standard deviations of price. Prices touching or exceeding these bands can indicate extreme overbought/oversold conditions. For instance, if SPY's price closes above the upper band, it may suggest an overbought market due for a pullback (sell signal), and below the lower band suggests oversold (buy signal). If we were to extend the strategy, Bollinger Band breakouts could be used similar to RSI to confirm or filter signals (e.g., only go long if price is below the lower band and other signals are bullish). Although Bollinger Bands were not explicitly used in our results, we mention them as a potential enhancement for capturing volatility extremes.

Signal Generation from Multiple Agents: Rather than trading directly on any single indicator, our strategy employs **multiple agents to generate signals** which are then aggregated. The agents and their signal outputs are:

- **Macro Sentiment Agent (VIX, Yields, Sector rotation)**: This agent looks at cross-market sentiment indicators. We wrote a custom function (called sentiment_signal in the code) that creates a "Trade Signal" based on daily changes in VIX, the tech-versus-financials ratio, and the 10-year yield:

    - If the **VIX spikes up** by more than a threshold (we used +5% daily change), it's a bearish sign (market fear rising) => subtract 1 from a sentiment score.

    - If VIX drops significantly (>5% down, indicating complacency or decreased fear), that's bullish => add 1 to the score.

    - If **Tech (XLK) outperforms Financials (XLF)** (the XLK/XLF ratio's daily change > 0), we interpret that as a bullish risk-on signal (investors favoring growth/tech) => add 1.

    - If Tech underperforms Financials (ratio < 0 change), that's relatively bearish (shift to defensive sectors) => subtract 1.

- If **10-year yield rises** sharply (>+0.01 in yield, i.e., 1%+ relative increase), that can be bearish for stocks (higher rates) => subtract 1.
- If yields fall significantly (>–0.01), that's bullish for stocks (lower rates) => add 1.

The sentiment score from these factors is then summed. If the final score > 0, the macro agent issues a **"Buy" signal**; if ≤ 0, it issues a **"Sell" signal**. This became our initial "Trade Signal" column in the data. Essentially, this agent is contrarian in nature – e.g., on a day when VIX plunges and yields drop (good for stocks), it likely signals Buy; on a day of a volatility spike or rate hike fears, it signals Sell.

- **News Sentiment Agent**: This agent uses the daily **news sentiment_score** described earlier. We created a DataFrame daily_sentiment aggregating news by date, and assigned a **Sentiment_Signal = "Buy" if sentiment_score > 0, else "Sell"**. This means if overall news tone in a day is positive (perhaps headlines about strong earnings, optimistic forecasts, etc.), the news agent votes to go long; if news tone is negative (recession fears, geopolitical risk, etc.), it votes to sell/short. The news agent's signal tends to be slower-moving (since sentiment may persist for days) but can occasionally react to major events that technicals haven't yet reflected.

- **Economic/Machine Learning Agent**: We developed a predictive model incorporating **economic indicators and technicals** to forecast the market's next move. We merged the economic data (unemployment, rates, CPI, GDP) with technical features (RSI, MACD, SMA, and also SPY's close price) into a single dataset (df_merged). We then computed each day's **future return** (next day's close minus current close) and defined a binary **target signal**: 1 if the next day's return was positive (stock went up), 0 if it was flat or negative. Using this labeled data, we trained an **XGBoost classifier** (XGBClassifier) to predict the next-day direction (up or down) from the features. The features used included current values of unemployment, interest rate, CPI, GDP, RSI, MACD, and SMA (so this model mixes macro and technical inputs). After training on historical data, we used the model to predict a **'predicted_signal'** for each day in our dataset (essentially an in-sample prediction to generate a signal series). The predicted_signal was 1 for expected rise and 0 for expected not-rise. We mapped this to a **recommendation**: 1 -> "Buy", 0 -> "Sell". This became our **Economic_Signal** (though it's generated by an ML model, many of its inputs are economic, so we treat it as the economic agent's voice). In effect, this agent looks at the broader fundamental picture and recent technical state to say whether the market is likely to go up or down tomorrow. Its perspective complements the purely technical rules and the news sentiment by providing a data-driven "forecast" element.

- **Technical Rules Agent**: In addition to the ML model's use of technical features, we also implemented a simpler rule-based check on technical conditions as an agent. Specifically, we defined a function to generate a **Technical_Signal** using predicted

returns from the model in combination with RSI thresholds. Pseudocode for this logic:

```
def generate_signal(row):
    if row['Predicted_Returns'] > UPPER_THRESHOLD and row['RSI'] < 30:
        return 'Buy'
    elif row['Predicted_Returns'] < LOWER_THRESHOLD and row['RSI'] > 70:
        return 'Sell'
    else:
        return None  # or neutral
```

- In practice, Predicted_Returns could be the probability or magnitude from the model. We chose thresholds to decide when a predicted move is strong enough to act on. We required alignment with RSI extremes: only go long if the model expects a significant upward move *and* RSI indicates oversold (increasing confidence the move will materialize), and vice versa for short. This Technical_Signal acts as a more conservative agent that might sometimes abstain (None) if conditions aren't extreme. In our combined signal table, this appears as the "Technical_Signal" column. Often early in the year it was NaN when conditions didn't trigger, whereas later when predictions and RSI lined up, it would register a Buy or Sell.

After computing all these, we had **four daily signal columns**:

- **Trade Signal** (from macro sentiment agent),
- **Technical_Signal** (from the technical rules agent),
- **Sentiment_Signal** (from news),
- **Economic_Signal** (from ML econ/technical model).

Signal Combination (Final Signal): To decide the actual trading action, we combined the above signals via a simple **majority voting system**. We created a function combine_signals(row) that examines the four agent signals for the day. It counts how many agents are signaling "Buy". If **at least 2 out of 4 agents vote Buy, the Final_Signal for that day is set to "Buy"**. Otherwise (meaning at least 3 are Sell or majority Sell), we set Final_Signal to "Sell". In case of an even split (2 Buy vs 2 Sell), our tie-break rule >=2 => Buy means it will still classify as Buy. This bias was intentionally slightly tilted bullish in ties, under the rationale that markets have a general upward drift and we preferred to be long absent a clear majority either way. The result of this combination is a single **Final_Signal each day: "Buy" or "Sell"** (we did not include a "Hold" or neutral final option – each trading week we either take a long or short stance based on this signal).

For example, on a given date, if the agents produced [Trade Signal = Sell, Technical_Signal = Buy, Sentiment_Signal = Buy, Economic_Signal = Buy], then Buy votes = 3, Sell votes = 1, so Final_Signal = **Buy**. Conversely, if two or more agents agree on Sell (and fewer than 2 on Buy), Final becomes Sell. This voting approach ensures no single faulty indicator can dictate a trade; it requires a consensus (or at least multiple confirmations). It is effectively implementing an **ensemble decision** from our multi-agent system.

Risk Management – Position Sizing and Stops: Generating a good signal is only part of a trading strategy – managing risk on each trade is crucial. Our strategy employs strict rules for **position sizing and stop-loss:**

- **Position Sizing:** We do not allocate the entire portfolio to a single trade. We introduced a parameter position_size (as a percentage of portfolio value per trade). In our tests, we tried values like 20%, 30%, 40%, up to 50%. The best results came with a **50% position size**, meaning the strategy uses half of its capital for each trade. This leaves the other half in cash, reducing risk exposure and allowing buffer for drawdowns. For example, if the portfolio value is $100,000 and a Buy signal occurs, the strategy will buy $50,000 worth of SPY shares (i.e., 50% of capital). Using a fractional position also implicitly caps the impact of any single trade's loss – even a total loss of the position would only drop the portfolio by 50%. In practice, losses are much smaller due to the stop-loss.

- **Stop-Loss:** We implemented a **fixed stop-loss percentage** on each trade via a strategy parameter. We tested various stop-loss levels (e.g., 3%, 5%, 10% etc. relative to entry price). The optimal setting in backtesting was a **3% stop-loss**. This means if the trade moves more than 3% against our entry price, we immediately exit to cut the loss. For a long position, if price falls 3% below entry, we sell to stop out; for a short position, if price rises 3% above entry, we buy-to-cover to stop out. Given our 50% position size, a 3% adverse move would reduce total portfolio value by about 1.5% (0.5 * 3%), which is a tolerable single-trade loss. This stop is implemented in the strategy logic to check each day's closing price against the entry price.

- **Time-Based Exit (Weekly Rebalancing):** Another safeguard is that the strategy does **not hold positions for more than a fixed number of days (hold_days)**. We set hold_days = 5 trading days (approximately one week). This means even if a trade is not stopped out, we will exit at the end of the week regardless. The idea is to **reset positions each week** to incorporate fresh signals and avoid lingering in a trade too long. It also limits overnight/weekend exposure to one week at most. In practice, this was coded by tracking the bar of entry and comparing with current bar index – if 5 or more trading days have passed, we close the trade on that fifth day's close. This rule locks in profits or losses at a regular interval and ensures the strategy frequently recalibrates to new information weekly.

- **No Multiple Simultaneous Positions:** The strategy only holds **one position at a time** (either long, short, or fully in cash between trades). We do not pyramid or hold long and short simultaneously. Each Monday (if not already in a trade), we open a new position according to the signal. If a stop triggers mid-week, we stay out until the next Monday. This rule simplifies risk tracking and ensures clear exposure at any time.

- **Commission & Slippage:** In backtesting, we included a small commission cost of 0.05% per trade (0.0005 in fractional terms) to simulate realistic trading friction. This has minimal impact on results given low trade frequency, but it slightly reduces returns and Sharpe to be more realistic. We did not explicitly model slippage (assuming we execute near the day's close price for simplicity), but since we trade very liquid SPY and only weekly, slippage would likely be minimal.

Taken together, these risk management measures kept the strategy's drawdowns very low. Every trade's downside was capped (~1.5% max portfolio loss if stop hit on a half position). The frequent rebalancing prevented any single bad signal from compounding losses over a long period. Position sizing also means if one week's signals were completely wrong, the portfolio could recover in subsequent weeks rather than being wiped out. As we'll see in results, the max drawdown observed was under 2%, confirming the effectiveness of these safeguards.

Backtesting Framework

To evaluate the strategy, we built a backtesting framework using the **Backtrader** library. The framework simulates our trading rules on historical data and records performance metrics. Key aspects of our backtesting methodology include:

In-Sample vs Out-of-Sample Split: We divided the data into a training (in-sample) period and a testing (out-of-sample) period to **avoid overfitting and data leakage**. We used roughly **90% of the data for in-sample development** and **reserved the final 10% for out-of-sample evaluation**. In practice, with about one year of data (approx 250 trading days), this meant using Jan 2024 through roughly December 2024 as the in-sample period, and then January 2025 into early February 2025 as the out-of-sample. All strategy parameters (like position size, stop-loss) were tuned using only the in-sample results. The final performance presented includes the out-of-sample to demonstrate how the strategy works on unseen data. This split ensures that our optimization (for example, choosing 50% and 3% as the best combo) is based on past data, and then we check if those choices still hold up reasonably well on new data beyond the training horizon.

Backtesting Logic Implementation: We encoded the trading strategy in a Backtrader Strategy class (called WeeklyStrategy). This class encapsulated the rules described in the Strategy Agent Module:

- In the next() method of the strategy, we check if we currently have a position. If yes, we evaluate exit conditions: either the stop-loss condition (price drop/rise beyond

threshold from entry) or the holding period condition (position open for >=5 days). If any exit condition is met, we execute a self.close() to exit the position at the current day's close.

- If we do not currently have a position (and therefore are in cash), we check if today is Monday (to enforce weekly entry timing). If it's Monday, we read the Final_Signal for that day (which Backtrader provides via our custom data feed that includes the signal). If Final_Signal is 1 ("Buy"), we issue a buy order for the position_size fraction of the portfolio. If Final_Signal is -1 ("Sell"), we issue a sell (short) order of that size. We then record the entry price and entry day to manage exits later.

By structuring the strategy this way, the backtest simulates exactly the intended behavior: only Monday entries, single position at a time, and monitored stops/timeouts for exits. We also created a custom Backtrader data feed to pass in our precomputed Final_Signal as part of the data (so the strategy can use it as if it were a market indicator). This avoids any forward-looking bias – the Final_Signal on each day is computed only from that day or prior information and is made available to the strategy on that day.

Safeguards Against Data Leakage: We took care that **no future data is used in generating signals for the current day**. For example, the economic ML model that predicted signals was trained on in-sample data and then used to predict in-sample as well as out-of-sample, but importantly, for out-of-sample days the model does not see those future days' outcomes (it's effectively making a genuine forward prediction). The majority voting signal for each day is based on that day's news, that day's macro data, and technicals up to that day. In code, we split the dataset into in-sample and out-of-sample subsets, ran the optimization on in-sample only, and only after fixing the best parameters did we run the strategy through the full period to log performance. This procedure imitates how we would use the strategy in real life: train/tune on past data, then deploy on new data. By not peeking into the out-of-sample period during tuning, we avoid overly optimistic results.

One point to note is that our machine learning agent was trained on the full in-sample (Jan–Dec 2024) and also evaluated in what effectively is an in-sample manner for those dates. This means the predicted signals for 2024 are somewhat fit to that period. However, since the final strategy signals also included other agents and since we ultimately validate on 2025 data (where the model had to generalize), we mitigated the risk of the ML component overfitting. Additionally, the stop-loss and weekly reset rules are straightforward and not something that can leak future info – they purely react to current or past price and time.

Metric Calculations: After running the backtest, we computed a suite of performance metrics to assess the strategy:

- **Sharpe Ratio:** A measure of risk-adjusted return, calculated as annualized average return divided by annualized volatility of returns. We computed daily portfolio returns from the backtest and then Sharpe = (mean daily return / std daily return) * $\sqrt{252}$. The resulting Sharpe for the strategy was **~1.20** (approximately 1.19 in our

best configuration). This indicates the strategy generated returns about 1.2 times the portfolio's volatility on an annual basis. (For context, a Sharpe > 1 is considered good in finance; the higher, the better risk-adjusted performance).

- **Max Drawdown:** The maximum peak-to-trough decline in the portfolio value during the period, expressed as a percentage. Our strategy's **Max Drawdown was about -1.92%**, meaning at worst it lost 1.92% from its highest point before recovering. This very low drawdown reflects the strategy's capital preservation focus. By contrast, SPY's buy-and-hold max drawdown in the same period was around -8.4%. A low drawdown is valuable because it implies the portfolio never fell far from its highs, which would give an investor confidence and shorter recovery times after any losses.

- **Win Rate:** The percentage of individual trades that were profitable. The strategy had a **Win Rate of ~53.6%**, winning slightly more than half of its trades. This is fairly typical for a trend-following or swing strategy – a win rate around 50-60% is acceptable as long as losses are cut short and winners are allowed to run (our strategy's risk management ensured losers were small). A 53.6% win rate indicates a slight edge in signal accuracy. (In our case, out of 28 trades, roughly 15 were wins and 13 were losses.)

- **Total Return:** The net percentage return on the portfolio over the entire backtest period. Starting from $100,000 on Jan 2, 2024, the strategy ended with about $106,984 by early Feb 2025, which is a **Total Return of +6.75%**. This is the growth achieved after all trading costs. While positive, this return is modest compared to simply holding SPY (which was an exceptionally strong period). It demonstrates the system's conservative nature – it gained steadily but didn't chase the full market rally.

- **Cumulative Returns & Portfolio Value:** We also tracked the daily portfolio value over time to examine the equity curve. The final portfolio value as noted was ~$106.98K from a $100K start. The path to this end value was quite smooth, with only minor dips along the way.

Table 1 below summarizes key performance metrics of the strategy:

Performance Metric	**Strategy Result**
Annualized Sharpe Ratio	1.19
Total Return (Mar 2024–Feb 2025)	+6.75%
Max Drawdown	–1.92%
Win Rate	53.6%

Final Portfolio Value	$106,983.99
Initial Portfolio Value	$100,000

Table 1: Summary of backtest performance metrics for the SPY Multi-Agent Strategy.

For comparison, the SPY buy-and-hold over the same period returned about +29.9% with a Sharpe ratio near 2.0 and max drawdown around 8.4%. This highlights that our strategy sacrificed upside in exchange for dramatically lower risk. Depending on an investor's objectives, this trade-off could be desirable (for risk-averse investors who prefer steady growth) or not (for those who simply want maximum return). Sharpe ratio being above 1 indicates the strategy was effective on a risk-adjusted basis, though not as effective as the benchmark in this particular bullish year for stocks.

Trade Analysis

Drilling down into the trade-by-trade behavior provides additional insight into how the strategy performed under various market conditions. Over the course of the backtest, the strategy executed **approximately 28 trades** (since it enters at most one trade per week, this number aligns with the number of weeks it was active). Each trade corresponds to a weekly position, either **long (Buy)** or **short (Sell)**, initiated on a Monday and closed by that Friday (or earlier if stopped out).

Trade Logs and Examples: Each trade is characterized by its entry date, direction, entry price, exit date, exit price, and profit or loss. Overall, profits on winning trades tended to be only slightly larger than losses on losing trades, but because winners outnumbered losers by a small margin, the net outcome was positive. With the 3% stop-loss, **no single loss exceeded 1.5% of the total portfolio**, and many losses were smaller. Winning trades were often in the +1% to +2% range (in portfolio terms), since the market did not move dramatically in a single week for most cases, and we capped the week's duration. Here are a few notable trade scenarios observed:

- **Early 2024 (Challenging period in uptrend):** In January 2024, the market was rallying strongly, but our multi-agent signals were initially cautious or bearish. For example, in the first week of January, macro sentiment was bearish (perhaps due to a spike in yields), and the final combined signal was "Sell". The strategy thus opened a short position at the beginning of that week. SPY continued to rise during the week, and by mid-week the loss hit the 3% stop-loss, causing the strategy to cover the short for about a 1.5% portfolio loss. This happened a few times in Q1 2024 – the system shorted into a rising market due to contrarian signals and took small losses. However, these losses were limited by the stop, and the strategy adapted as more agents turned bullish later.

- **Mid-2024 (Trend-following gains):** By the spring of 2024, technical and ML agents started flipping to "Buy" as the uptrend was well-established. For instance, on **April 29, 2024 (Monday)**, the combined signal was still "Sell" (perhaps news or macro

was bearish that day), so the strategy shorted. However, by the next week, signals had caught up – the first week of May saw more "Buy" votes. The strategy entered a long position in early May when Final_Signal turned "Buy". SPY's price rose that week, so that trade was closed with a gain of around +1%. In general, during steady upward weeks, the strategy's long trades yielded modest positive returns. Because of the weekly reset, it often captured the middle of multi-week moves in a stepwise fashion rather than compounding a single position.

- **Volatile/Sideways periods:** In choppier weeks, the strategy sometimes alternated positions. A notable phase was in August 2024 when the market had a brief pullback. One week, signals indicated "Sell" and the strategy shorted SPY. The market indeed fell that week, giving the strategy a winning short trade (one of the larger wins, as SPY dropped ~2% and the short position gained roughly +1% on the portfolio). The following week, signals switched to "Buy" as technicals showed oversold conditions and news perhaps improved – the strategy went long and caught the rebound for another small gain. These back-to-back trades demonstrate the system's ability to handle short-term reversals: the combination of agents correctly anticipated a downturn and then a recovery, resulting in consecutive profitable trades (one short, then one long).

- **Late 2024 (Market peak and decline):** The end of 2024 saw SPY reaching new highs and then a minor correction. Our strategy by this time had most agents in sync. Around mid-December 2024, SPY was at a peak; the economic ML agent and possibly news started to flash warnings (e.g., if news talked about overbought conditions or the ML model saw rising risks). The combined signal turned "Sell" near the top, and the strategy entered a short. Right afterwards, SPY had a modest pullback of a few percent into the end of December. The short trade during this week was quite successful – it hit its profit target (in this case, time-based exit on Friday, since we didn't use a profit-take limit, but it closed near the week's low). That trade brought the portfolio to its high watermark (~+8% from start). In early January 2025, the market rallied again sharply (SPY making new highs in the new year), and the strategy's agents flipped to Buy late (still carrying a short bias into the first week of January 2025, which resulted in a small loss as it shorted into a rally then stopped out). This sequence showed that while the strategy protected well against the late-year dip (it profited from it), it was a bit slow to rejoin the rally, missing some of the rebound.

By analyzing the trade log, we noticed **certain patterns**:

- The strategy **avoided catastrophic losses** entirely. Thanks to the stop-loss, every losing trade was cut off early. This kept drawdowns small and allowed the next trade to start from a relatively strong equity position.

- The **winning trades often occurred when multiple agents aligned** in the correct direction, especially when a clear trend or reversal was underway. For example,

when all four signals agreed (which did happen occasionally), those trades were usually successful. Conversely, trades taken on a slim majority (2 vs 2 tie resolved to Buy, or 2 vs 1 in cases one agent abstained) were more hit-or-miss.

- The system sometimes experienced **whipsaw** in rapidly changing weeks – e.g., a stop-out on a false move then the next week a correct signal. This is an inherent risk of a shorter-term strategy, but limiting trades to weekly and using confirmatory signals helped reduce whipsaw frequency.

Performance Trends Over Time: Over the year, the equity curve of the strategy rose gradually, in a stair-step manner. It had plateaus during extended strong bullish periods (since it often sat out or made only tiny gains while SPY climbed) and jumps mainly when capturing down moves or short-term swings. Specifically:

- **Jan–Mar 2024:** The portfolio dipped slightly below $100K (to about $98.5K at its lowest) due to consecutive small short-trade losses in the face of a relentless rally. During this time, the strategy was preserving capital (a ~1.5% drawdown vs SPY's +8% gain in the quarter).

- **Apr–Jul 2024:** The strategy recovered and oscillated around breakeven to +2%. It caught some profitable weeks as signals improved. By mid-year it was up a couple percent.

- **Aug–Oct 2024:** A volatile late summer benefited the strategy; it started climbing to new equity highs (around +4% by October) as it successfully traded both sides of the market volatility (this period had several wins as described).

- **Nov–Dec 2024:** The portfolio surged to about +8% thanks to timely short positions during a minor correction. This was the peak performance.

- **Jan 2025:** Gave back a bit of profit (down to +6.7%) as the strategy's contrarian stance led to a small loss when the market ripped upward. However, once that move was digested, the strategy again ended on a positive note in early Feb 2025 as it closed the final trades.

The end result is that the **strategy's return curve was much smoother than SPY's**. While an SPY holder would see big swings (up nearly 30% but with some 5-8% pullbacks in between), our strategy's owner saw a very mild growth with hardly any swings exceeding 2%. Psychologically, this can be easier to handle, though the opportunity cost in a bull market is missing out on large gains.

Performance Dashboard

To present the results and enable interactive exploration, we built a Streamlit dashboard. The dashboard provides a visual summary of the strategy's performance compared to a baseline of SPY buy-and-hold.

*Figure 1: Performance dashboard for the SPY Multi-Agent Trading System. The top panels show key performance metrics (Sharpe Ratio, Max Drawdown, Win Rate, Total Return), and the bottom panel plots the equity curve of the **Agent Strategy (green line)** versus **SPY Buy & Hold (orange dashed line)** over time.*

As shown in *Figure 1*, the dashboard highlights several important points:

- The **metrics section** at the top displays the final values of our performance metrics: **Sharpe Ratio ~1.19**, **Max Drawdown ~1.92%**, **Win Rate ~53.57%**, and **Total Return ~4.70%** in that particular screenshot. *(Note: The dashboard screenshot reflects an earlier run or slightly different parameter, showing 4.70% total return – for the optimized run we report 6.75%. The differences are likely due to parameter tuning; the best config yielded 6.75% total return and Sharpe 1.19.)* These metrics allow quick evaluation of the strategy's risk vs reward. For instance, the Sharpe 1.19 is labeled in green indicating a decent risk-adjusted performance; the drawdown of

1.92% is extremely low (a strong positive); the win rate around 53.6% confirms just over half the trades succeeded; and total return of a few percent shows modest growth.

- The **final portfolio value** is also shown (in the dashboard it's around $104,813 for that run, versus $100,000 start). This gives the user an immediate sense of how much the strategy earned in absolute terms.

- The **equity curve plot** compares the strategy (green solid line) with a **benchmark SPY buy-and-hold (orange dashed line)**. We can see that the SPY (orange) climbed dramatically over the year, ending much higher than the strategy's green line. However, the SPY line has visible dips (volatility), whereas the green line is much smoother. For example, around late March 2024, SPY dips while our strategy hardly moves, implying it was likely in cash or a correct hedged position. The divergence grows larger in late 2024 when SPY surges (orange goes up steeply) but our strategy (green) rises only gently. By early 2025, SPY's slight downturn doesn't hurt our strategy much at all (green line even ticks up when orange ticks down, due to a profitable short trade at that time). This chart succinctly demonstrates the **risk/return trade-off**: the strategy avoided the big swings but also didn't capture the big gains. An interactive version of this plot allows hovering to see exact values by date, which can be useful to pinpoint when the largest gains or losses occurred for each approach.

- The dashboard also includes a download button for the underlying data so that one can further analyze or verify the performance metrics externally if desired.

Overall, the dashboard provides an accessible summary for both technical and non-technical stakeholders. One can see at a glance that the multi-agent strategy was very conservative (low drawdown, moderate Sharpe) and can contrast its performance with the benchmark visually. The use of green vs orange color coding helps to easily identify our strategy versus the market. Such a tool is valuable in explaining the strategy's behavior to others: for instance, an investor might ask why the strategy underperformed the market – the chart and metrics help show that it was a conscious choice for lower risk, as evidenced by the Sharpe and drawdown differences.

Strategic Reflections

Building and evaluating the SPY Multi-Agent Trading System yielded several insights, along with challenges encountered along the way:

What Worked Well:

- **Risk Management and Stability:** The combination of moderate position sizing and strict stop-loss proved very effective. The strategy never experienced a deep drawdown, which is a big success in terms of capital preservation. This validates the design choice to emphasize risk-adjusted returns. Even when some signals were

wrong (which is inevitable), the losses were contained and did not derail the overall performance. The weekly reset also worked well to regularly lock in profits and prevent small losses from growing.

- **Multi-Agent Signal Approach:** Using multiple sources of information (technical, news, economic) added robustness. Each agent brought a different perspective: technicals captured market momentum, news reflected real-world events and sentiment, and economic indicators provided a fundamental backdrop. We observed that no single agent dominated the signals at all times – at different points, different agents were the deciding factor in the Final Signal. This indicates our approach was adaptive: e.g., during a calm uptrend, technical signals and ML predictions might drive the decisions, whereas during a sudden news-driven event, the news sentiment agent might tip the balance. The majority vote logic smoothed out noise – a random false signal from one source rarely triggered a bad trade unless others agreed. Overall, this ensemble method improved the **consistency** of signals.

- **Modular Design & Clarity:** Separating data collection from strategy logic greatly streamlined development. We could iteratively improve data quality (for example, adjust how we calculated sentiment or add an indicator) without touching the backtest code. Similarly, we could tweak strategy rules without worrying about data issues. This modular approach saved time and made debugging easier. For instance, when the strategy wasn't performing initially, we could quickly see if it was due to poor signals (analysis agent issue) or execution (backtester issue).

- **Identification of Market Regime:** The multi-agent system, in a way, was able to **identify market regimes**. When all signals aligned (which often coincided with either strong bullish consensus or strong bearish consensus in the world), the strategy capitalized well. When signals conflicted, it often meant the market was in a more uncertain or transitioning phase; in those times the strategy tended to take smaller positions (effectively half in cash always, and sometimes quickly stopped out) which is appropriate in choppy conditions. This dynamic adjustment according to regime was an implicit benefit of combining different indicators.

Challenges and How They Were Addressed:

- **Underperformance in Strong Trending Market:** The glaring challenge was that our strategy significantly underperformed a simple long SPY strategy in this particular period (a strong bull market). The conservative posture and contrarian signals meant we often **missed upside**. This is a classic challenge in developing a more complex strategy: balancing risk vs reward. We recognized that our parameters (50% allocation, 3% stop) were tuned to prioritize lower drawdowns, at the expense of higher returns. Addressing this is tricky – if we loosen stops or increase position size to chase return, we risk larger drawdowns. For this project, we accepted the trade-off, but in the future, one might incorporate trend-following elements (e.g., a rule that if the market is in a sustained uptrend, perhaps put more

weight on technical trend signals and less on contrarian macro signals) to better capture long runs.

- **Data Alignment and Quality:** Combining multiple data sources (Yahoo Finance for prices, an external file for news, FRED for economics) introduced some data management challenges. For example, ensuring that the news we had corresponded exactly to our trading dates (we had to aggregate by date and join, making sure no off-market days sneaked in). Another example was the economic data frequency mismatch; interpolating GDP or CPI on a daily basis could introduce spurious precision. We addressed these by mostly relying on daily percent changes or threshold triggers rather than raw levels (so slight interpolation errors wouldn't flip a signal unless they passed a threshold). We also thoroughly time-aligned all series to market dates and dropped days where data might be incomplete. This required careful processing in the data collection notebook, but once done, it allowed seamless merging. Testing the data agent output by printing tail/head of dataframes helped verify alignment (we printed and inspected a combined table to ensure all columns made sense together).

- **Model Overfitting Concerns:** Using an ML model (XGBoost) within the strategy brought up the risk of overfitting. We trained on a relatively short period (2024 data) and also used those predictions in the same period. This means the model could have inadvertently learned noise (especially with so many features relative to ~200 data points). The evidence was that the model's recommendation column often flipped day-to-day in some periods, suggesting it might be somewhat overfit to daily wiggles. To mitigate this, we constrained the model complexity (setting a max_depth of 4 and limited estimators) and noticed that the model's signal alone didn't dominate the final decision – it was just one vote. Additionally, our out-of-sample test in Jan 2025 gave us confidence: the strategy still made a bit of profit in Jan 2025, which suggests the model's signals did not completely break down on new data. In future iterations, one could improve this by gathering more training data (e.g., use multiple years for the ML model) or by using cross-validation to refine it. For the scope of this project, keeping the ML influence modest (just one agent among four) was a practical solution to avoid overfitting disasters.

- **Complexity of Tuning Multiple Components:** With many moving parts (thresholds for VIX, yields, stop-loss levels, etc.), tuning had to be done carefully. We used a brute-force/iterative search for the two most sensitive parameters (position_size and stop_loss) while keeping others reasonable by domain knowledge (e.g., VIX 5% threshold was chosen based on typical volatility swings). One challenge was that some parameters are interdependent – for example, if we increased hold_days beyond 5, maybe certain trades would have become big winners in the trending market, altering the optimal stop. We decided to stick with a straightforward weekly cadence for simplicity. We also considered tie-break bias (Buy on ties) – if we had instead chosen Sell on ties or "no trade" on ties, results

would differ. We tested a neutral stance on ties (i.e., require >2 buys to go long, >2 sells to go short, otherwise no trade) in some runs, and found that our approach of forcing a direction (with slight bias to buy) kept the system engaged regularly and didn't harm performance badly. The challenge remains to systematically tune all such logic; our approach was heuristic and could be improved with more exhaustive search or optimization algorithms given more time.

Suggested Improvements:

- **Introduce a Neutral Position Option:** One clear improvement would be to allow the strategy to **stay in cash (no position) if signals are very mixed**. Currently, we force a long or short every week. If we had a "Hold" signal when votes are tied or confidence is low, we might avoid some whipsaw trades and save on transaction costs. This could be implemented by requiring, say, at least a 3-1 vote to take a position, otherwise skip that week. It would likely reduce trade count further and possibly improve Sharpe (though it might lower total return if the market moved that week).

- **Dynamic Position Sizing:** Our fixed 50% allocation is static. We could refine this by making the position size **proportional to the strength of signals**. For example, if all 4 agents strongly say Buy, perhaps allocate 70% that week; if only 2 say Buy vs 2 Sell (tie), maybe allocate just 20% or even 0%. This risk-adjusted sizing could mean the strategy presses its advantage when there is high consensus and pulls back when uncertain. This would require calibrating how to map signal consensus or model confidence to a position fraction.

- **Improve Trend Capture:** As discussed, adding a trend-following mechanism could help in long bull markets. One idea: include a rule that if SPY has been above its moving average for X weeks and macro conditions are not strongly bearish, default to a long bias (or require a higher threshold to go short). This kind of regime filter could prevent the strategy from shorting in environments where it's fighting a strong trend. Our multi-agent signals did this implicitly to some degree (as technical and ML eventually adjust), but an explicit filter might react faster.

- **Extend ML Model Training:** The XGBoost model could be trained on a longer history (if data from 2010s, etc., were available) to improve its robustness. Also, using more features like momentum of economic indicators or even lagged signals could enhance its predictive power. Alternatively, one could explore a simpler model (or none at all) if it's not adding value – in our results it's hard to isolate the ML agent's contribution, so analyzing feature importance or running ablation tests (removing one agent to see impact) would be an insightful next step.

- **Additional Data Sources:** We could incorporate other agents or data to further improve decisions. For example, a **seasonality agent** (accounting for calendar effects like "January effect" or option expiration weeks) could provide another small

edge. Or a **sentiment agent** using social media sentiment might complement the news agent. We could also add **Bollinger Band signals** as mentioned, or volume-based indicators, etc. Any new agent should ideally bring orthogonal information so as to genuinely enhance the ensemble.

- **Real-time Implementation Considerations:** If deploying this strategy live, we'd refine the execution – e.g., deciding whether to enter at Monday open vs close, how to set stop-loss orders intraday (backtester used closing prices for simplicity, but live trading would use intraday stops). We might use limit orders for entry to avoid poor fills if Monday opens with a gap, etc. These practical tweaks could improve actual performance vs theoretical backtest.

Executing the code –

1. Execute the 'Data Collection Agent file' in Jupyter Notebook - This file will execute the code and store all the required data in a processed format.
2. Next, execute the 'Analysis and Strategy Agent file in Jupyter Notebook – This file will execute the code and make all the analysis agents work together, give a combined signal and execute the strategy with the portfolio risk measurement metrics of SPY.
3. Once the file is run, go to Anaconda prompt and run the command = 'streamlit run dashboard.py' in the folder where you have saved the files, this will make sure that the dashboard is displayed.

Conclusion: The SPY Multi-Agent Trading System succeeded in creating a **balanced, risk-conscious strategy** through the synergy of multiple analytical agents. It met their primary objective of protecting capital and achieving steady growth, though at the cost of underperforming in a runaway bull market. The exercise underscored that **no single strategy is perfect for all market conditions** – our multi-agent approach handled volatility and downturns well, but a pure trend strategy would beat it in a relentless rally. The key insight is that **diversification of signals can yield a smoother equity curve**, reducing risk. Going forward, adjusting the blend of agents (or their decision rules) could shift the strategy along the risk-reward spectrum as desired. This project demonstrated the workflow of designing a complex trading system: from data gathering, feature engineering, strategy rule coding, to evaluation and visualization. Each step provided learning opportunities, from technical (Python backtrader usage, data handling) to analytical (financial indicator interpretation, the impact of news/econ on markets). The resulting system is a solid foundation that could be iteratively improved and tuned with additional data and more sophisticated techniques, bridging the gap between a classroom challenge and a real quantitative trading strategy.

Appendix

Key Code Snippets:

Below we include some snippets of the implementation to supplement the report.

- *Multi-Agent Signal Combination:* This code shows how the four agent signals are combined into the final trading signal using a majority vote.

```
def combine_signals(row):
    signals = [row['Trade Signal'], row['Technical_Signal'],
               row['Sentiment_Signal'], row['Economic_Signal']]
    buy_count = signals.count('Buy') + signals.count('BUY')  # count
both 'Buy' and uppercase 'BUY'
    return 'Buy' if buy_count >= 2 else 'Sell'

# Apply combination on daily signals DataFrame
daily_signals['Final_Signal'] = daily_signals.apply(combine_signals,
axis=1)
```

Comment: Here daily_signals is a DataFrame containing columns for each agent's recommended signal per day. The function counts how many agents voted "Buy". If 2, 3, or 4 agents vote Buy, the Final_Signal is set to "Buy". Otherwise (which implies buy_count < 2, meaning majority are Sell), Final_Signal is "Sell". This voting scheme ensures a tie (2 vs 2) yields Buy.

- *Trading Strategy Class (Backtrader):* This snippet outlines the WeeklyStrategy class which enforces the weekly entry/exit and stop-loss rules.

```
class WeeklyStrategy(bt.Strategy):
    params = dict(
        stop_loss=0.03,       # 3% stop-loss
        hold_days=5,          # max holding period
        position_size=0.5     # 50% of portfolio per trade
    )
    def __init__(self):
            self.signal = self.datas[0].signal  # access the
'Final_Signal' line from data feed
        self.entry_price = None
        self.entry_bar = None

    def next(self):
        date = self.datas[0].datetime.date(0)
        if self.position:  # already in a trade
            # Check exit conditions
            holding_period = len(self) - self.entry_bar
```

```
                if (self.data.close[0] <= self.entry_price * (1 -
self.params.stop_loss)) or \
               (holding_period >= self.params.hold_days):
                 self.close()  # exit position (either stop-loss hit
or time is up)
        else:
            # No current position, decide on Monday entries
            if date.weekday() == 0:  # Monday
                    if self.signal[0] == 1:         # Final_Signal
indicates "Buy"
                                size = (self.broker.getvalue() *
self.params.position_size) / self.data.close[0]
                    self.buy(size=size)
                    self.entry_price = self.data.close[0]
                    self.entry_bar = len(self)
                     elif self.signal[0] == -1:     # Final_Signal
indicates "Sell"
                                size = (self.broker.getvalue() *
self.params.position_size) / self.data.close[0]
                    self.sell(size=size)
                    self.entry_price = self.data.close[0]
                    self.entry_bar = len(self)
```

Comment: self.datas[0].signal is the Final_Signal for the current day (Backtrader feeds allow adding custom data lines). We map "Buy" to +1 and "Sell" to -1 when preparing the data feed, so here signal[0] == 1 corresponds to a Buy signal. On each bar (day), if a position exists, we compute how many days we've held (holding_period) and compare current price to entry_price for stop-loss. If stop-loss or 5-day limit is reached, self.close() closes the trade. If no position and it's Monday, we enter according to the signal, using order_target_percent implicitly by calculating size of shares equal to 50% of current portfolio value. This code ensures exactly the behavior described in the strategy: weekly entry on signal, with predetermined position size and exits.

- *Technical Indicator Calculation:* The data collection agent used the pandas_ta library to compute indicators. For example:

```
import pandas_ta as ta

# Assuming spy_df is a DataFrame with SPY price data indexed by date
spy_df['RSI'] = ta.rsi(spy_df['Close'], length=14)
spy_df['SMA'] = ta.sma(spy_df['Close'], length=14)
macd_df = ta.macd(spy_df['Close'])  # returns a DataFrame with MACD,
MACD_signal, MACD_hist
spy_df['MACD'] = macd_df['MACD_12_26_9']
```

This produces the 14-day RSI, 14-day SMA, and MACD (12,26,9) line for SPY, which we then saved to technicaldata.csv for use by the strategy agent.

- *Dashboard Code Snippet:* (for completeness, showing how the results are presented in Streamlit)

```
import streamlit as st
import pandas as pd
import plotly.graph_objects as go

# Load performance data
results_df = pd.read_csv("results_df.csv")  # contains metrics for various params
daily_df = pd.read_csv("daily_portfolio_comparison.csv", parse_dates=["Date"]).set_index("Date")

# Select best param configuration
best_config = results_df.sort_values(by=['Sharpe_Ratio','Total_Return(%)'], ascending=False).iloc[0]
sharpe_ratio = best_config['Sharpe_Ratio']
max_drawdown = best_config['Max_Drawdown(%)']
win_rate = best_config['Win_Rate(%)']
total_return = best_config['Total_Return(%)']
final_value = best_config['Final_Portfolio_Value']

# Display metrics
st.metric("Sharpe Ratio", f"{sharpe_ratio:.2f}")
st.metric("Max Drawdown", f"{max_drawdown:.2f}%")
st.metric("Win Rate", f"{win_rate:.2f}%")
st.metric("Total Return", f"{total_return:.2f}%")
st.success(f"Final Portfolio Value: ${final_value:,.2f}")

# Plot equity curves
fig = go.Figure()
fig.add_trace(go.Scatter(x=daily_df.index, y=daily_df['AP_Value'], name='Agent Strategy', line=dict(color='green')))
fig.add_trace(go.Scatter(x=daily_df.index, y=daily_df['SP_Values'], name='SPY Buy & Hold', line=dict(color='orange', dash='dash')))
fig.update_layout(title="Portfolio Comparison: Strategy vs SPY", yaxis_title="Portfolio Value ($)")
st.plotly_chart(fig)
```

This code fetches the best run's stats and the daily portfolio values (Agent Portfolio vs SPY) and creates an interactive line chart and metric displays in the Streamlit app.

Documentation and References:

- Backtrader Documentation: used for implementing the strategy class and analyzers. (See Backtrader's official docs for Strategy and Analyzer usage.)
- pandas_ta Documentation: for technical indicator functions like ta.rsi, ta.macd, etc.
- Yahoo Finance (yfinance) API: used to retrieve historical price and VIX data.
- FRED API (Fred library): used to retrieve the 10-year yield series.
- News data source: provided as newsdata.csv (assumed to be compiled from financial news sites).
- XGBoost Documentation: for the XGBClassifier parameters and usage to ensure multi-class handling (though we used it in a binary classification context).

These references guided the implementation and ensure that our use of libraries and calculations conform to standard definitions (e.g., verifying that our Sharpe Ratio formula annualizes correctly, confirming that RSI period length is interpreted as expected, etc.). The combination of these tools allowed us to focus on strategy logic rather than low-level data handling.